

UNIVERSIDAD AUTÓNOMA DEL ESTADO DE MÉXICO

FACULTAD DE INGENIERÍA

UNIDAD DE APRENDIZAJE

Análisis del Sistema

Unidad 4

Conceptos fundamentales y ejemplos

Mtra. Beatriz Orozco Garduño

Mayo 2026

Introducción

El análisis del sistema constituye una de las fases más críticas en el ciclo de vida del desarrollo de software. Es la etapa donde el ingeniero descompone un problema complejo en partes comprensibles, identifica los requerimientos del cliente y modela la realidad del negocio mediante representaciones gráficas estandarizadas. Esta unidad presenta los conceptos, técnicas y herramientas necesarios para formular sistemas a partir de los requerimientos, utilizando UML como lenguaje universal de modelado.

Los temas tratados en este documento permiten al estudiante adquirir competencias para visualizar la estructura y comportamiento de un sistema antes de su implementación, lo cual reduce errores, optimiza recursos y facilita la comunicación entre los miembros del equipo de desarrollo.

Objetivo de la unidad

Formular problemas o requerimientos del sistema en componentes más pequeños y comprensibles utilizando UML, con el fin de visualizar y estructurar claramente los componentes, interacciones y relaciones del sistema.

4.1. Análisis de Sistemas

Concepto fundamental

El análisis de sistemas es el proceso sistemático de estudiar una situación o problema con el propósito de identificar sus componentes, sus relaciones y la forma en que estos interactúan para producir un resultado. En ingeniería de software, esta actividad transforma necesidades del usuario en una especificación detallada que servirá como base para el diseño.

Definición formal

El análisis de sistemas consiste en la descomposición de un problema en partes elementales, el estudio de cada parte de manera individual, y la posterior reconstrucción de la solución mediante la integración de dichas partes en un todo coherente y funcional.

Objetivos del análisis de sistemas

- Comprender el dominio del problema y el contexto organizacional.
- Identificar a los usuarios, actores externos y reglas de negocio.
- Especificar requerimientos funcionales y no funcionales.
- Construir modelos abstractos que faciliten la comunicación.
- Validar que la solución propuesta satisface las necesidades reales.
- Detectar inconsistencias, omisiones y conflictos en los requerimientos.

Actividades principales

- 1. Recolección de información:** Mediante entrevistas, observación, cuestionarios y análisis de documentos existentes.
- 2. Identificación de requerimientos:** Determinar qué debe hacer el sistema (funcionales) y cómo debe hacerlo (no funcionales).
- 3. Modelado:** Construir representaciones gráficas y formales del sistema.
- 4. Especificación:** Documentar de forma precisa el comportamiento esperado.
- 5. Validación:** Verificar con el cliente que los modelos reflejan correctamente sus necesidades.

Ejemplo aplicado

Caso: Sistema de Control Escolar Universitario

Un departamento académico necesita automatizar el registro de calificaciones, inscripciones y generación de boletas. El analista del sistema debe entrevistar a coordinadores, profesores y personal administrativo; revisar los formatos actuales en papel; identificar los reglamentos escolares aplicables; y modelar los procesos de inscripción, captura de calificaciones, generación de actas y consulta de historial académico. El resultado del análisis será un conjunto de documentos y diagramas que describen el sistema antes de escribir una sola línea de código.

Tipos de análisis

Tipo	Descripción	Cuándo aplicarlo
Estructurado	Se enfoca en los datos y procesos mediante DFD y diccionarios de datos.	Sistemas con flujos de información bien definidos.
Orientado a objetos	Modela el sistema como una colección de objetos que colaboran. Usa UML.	Sistemas complejos, evolutivos, con reutilización.
Orientado a aspectos	Separa las preocupaciones transversales (seguridad, logs) del núcleo.	Sistemas con requerimientos no funcionales transversales.
Ágil	Análisis iterativo, ligero, con historias de usuario.	Proyectos con requerimientos cambiantes.

4.2. UML (Unified Modeling Language)

Concepto fundamental

UML es un lenguaje gráfico estandarizado para visualizar, especificar, construir y documentar los artefactos de un sistema de software. Fue desarrollado por Grady Booch, James Rumbaugh e Ivar Jacobson, conocidos como los "tres amigos", y es actualmente mantenido por el Object Management Group (OMG). La versión vigente más utilizada es UML 2.5.

¿Por qué UML?

Antes de UML, cada metodología tenía su propia notación, lo que dificultaba la comunicación entre equipos. UML unificó las mejores prácticas de Booch, OMT y OOSE en un único lenguaje visual que cualquier ingeniero de software puede leer e interpretar.

Características principales

- Independiente del lenguaje de programación y de la plataforma.
- Independiente del proceso de desarrollo (puede usarse con RUP, Scrum, XP, etc.).
- Soporta tanto análisis como diseño orientado a objetos.
- Permite modelar aspectos estáticos (estructura) y dinámicos (comportamiento).
- Extensible mediante perfiles y estereotipos.

Clasificación de diagramas UML 2.5

UML 2.5 define 14 tipos de diagramas, organizados en dos grandes categorías:

Diagramas estructurales (estáticos)

Representan los elementos del sistema y sus relaciones. Muestran cómo está construido el sistema.

- Diagrama de clases
- Diagrama de objetos
- Diagrama de componentes
- Diagrama de despliegue
- Diagrama de paquetes
- Diagrama de estructura compuesta
- Diagrama de perfiles

Diagramas de comportamiento (dinámicos)

Representan cómo interactúan los elementos a lo largo del tiempo. Muestran qué hace el sistema.

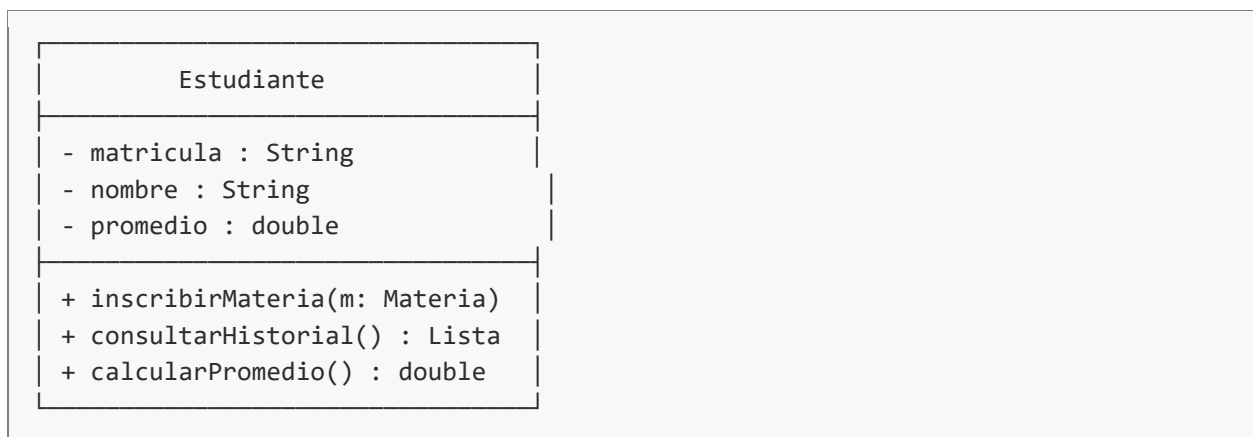
- Diagrama de casos de uso
- Diagrama de actividades
- Diagrama de estados
- Diagrama de secuencia
- Diagrama de comunicación
- Diagrama de tiempos
- Diagrama global de interacciones

Elementos comunes en UML

Elemento	Notación	Significado
Clase	Rectángulo con 3 secciones	Define un tipo de objeto con atributos y métodos.
Objeto	Rectángulo con nombre subrayado	Instancia concreta de una clase.
Actor	Figura humana (stick figure)	Entidad externa que interactúa con el sistema.
Caso de uso	Elipse con verbo en infinitivo	Funcionalidad observable desde fuera.
Asociación	Línea continua	Relación estructural entre dos elementos.
Herencia	Flecha con triángulo blanco	Una clase hija extiende a la padre.
Dependencia	Flecha discontinua	Un elemento depende de otro para funcionar.

Ejemplo de notación básica

Considere la siguiente clase Estudiante representada en UML:



Los símbolos de visibilidad son: + público, - privado, # protegido, ~ paquete.

4.3. Diagramas de Casos de Uso

Concepto fundamental

Un diagrama de casos de uso describe el comportamiento del sistema desde la perspectiva del usuario. Muestra qué hace el sistema y quién lo utiliza, sin entrar en detalles de cómo se implementa internamente. Es la herramienta más utilizada para capturar requerimientos funcionales.

Propósito

Documentar las funcionalidades del sistema que aportan valor a los actores externos. Cada caso de uso responde a la pregunta: ¿qué puede hacer un usuario con el sistema?

Elementos del diagrama

Actor: Rol que juega una entidad externa (persona, sistema, dispositivo) al interactuar con el sistema. Se representa con una figura humana.

Caso de uso: Funcionalidad observable que el sistema ofrece a los actores. Se nombra con un verbo en infinitivo. Se representa con una elipse.

Sistema (frontera): Rectángulo que delimita el alcance del sistema. Los casos de uso están dentro y los actores fuera.

Relaciones: Asociación (actor-caso de uso), inclusión («include»), extensión («extend»), generalización.

Tipos de relaciones entre casos de uso

Relación	Notación	Cuándo se usa
«include»	Flecha discontinua con etiqueta	El caso base SIEMPRE ejecuta el incluido. Ejemplo: "Realizar venta" incluye "Validar pago".
«extend»	Flecha discontinua con etiqueta	El caso extendido se ejecuta OPCIONALMENTE. Ejemplo: "Realizar venta" puede extenderse a "Aplicar descuento".
Generalización	Flecha con triángulo blanco	Un caso o actor hereda de otro. Ejemplo: "Pago con tarjeta" generaliza "Pago con débito".

Ejemplo completo: Sistema de Biblioteca Universitaria

Descripción del caso

Una biblioteca universitaria requiere un sistema para gestionar préstamos de libros. Los estudiantes pueden buscar libros, solicitar préstamos y renovarlos. Los bibliotecarios registran nuevos libros, gestionan devoluciones y consultan reportes. El sistema debe validar al usuario antes de cualquier operación.

Actores identificados:

- Estudiante (actor primario)
- Bibliotecario (actor primario)
- Sistema de Autenticación UAEMéx (actor secundario)

Casos de uso identificados:

- Buscar libro
- Solicitar préstamo
- Renovar préstamo
- Registrar libro
- Gestionar devolución
- Consultar reportes
- Validar usuario (incluido en todos los anteriores)
- Aplicar multa (extiende a Gestionar devolución)

Especificación textual de un caso de uso

Cada caso de uso debe documentarse con una plantilla detallada. A continuación se muestra el caso "Solicitar préstamo":

CASO DE USO: Solicitar préstamo

ACTOR PRIMARIO: Estudiante

PRECONDICIÓN: El estudiante está autenticado y no tiene multas pendientes.

POSTCONDICIÓN: Se registra el préstamo y se actualiza el inventario.

FLUJO PRINCIPAL:

1. El estudiante busca el libro deseado.
2. El sistema muestra la disponibilidad.
3. El estudiante confirma la solicitud.
4. El sistema verifica el límite de préstamos (máx. 3).
5. El sistema registra el préstamo con fecha de devolución (+14 días).
6. El sistema envía comprobante por correo.

FLUJOS ALTERNATIVOS:

- 4a. Si el estudiante alcanzó el límite: el sistema rechaza y notifica.
- 2a. Si el libro no está disponible: el sistema ofrece reservarlo.

4.4. Patrones de Análisis

Concepto fundamental

Un patrón de análisis es una plantilla reutilizable que captura una solución probada a un problema recurrente en el modelado conceptual de sistemas. A diferencia de los patrones de diseño (que abordan soluciones de implementación), los patrones de análisis modelan estructuras del dominio del negocio.

Definición de Martin Fowler

Los patrones de análisis son grupos de conceptos que representan una construcción común en el modelado de negocio. Pueden ser relevantes para un solo dominio o pueden abarcar muchos dominios.

Beneficios

- Aceleran el modelado al partir de soluciones probadas.
- Mejoran la calidad y comprensión del modelo conceptual.
- Facilitan la comunicación entre analistas mediante un vocabulario común.
- Reducen errores comunes en el análisis del dominio.
- Permiten reutilizar conocimiento del negocio entre proyectos.

Patrones clásicos de análisis (Fowler)

1. Patrón Party (Parte)

Modela personas u organizaciones de manera unificada. En muchos sistemas, una entidad puede ser una persona física o una organización, y ambas comparten propiedades comunes (nombre, dirección, contactos).

Ejemplo

En un sistema de facturación, un cliente puede ser una persona física (con CURP, RFC) o una empresa (con razón social, RFC). El patrón Party crea una superclase abstracta Party con subclases Persona y Organización.

2. Patrón Accountability (Responsabilidad)

Modela relaciones de autoridad y responsabilidad entre entidades. Útil para representar estructuras organizacionales, jerarquías y roles.

Ejemplo

En un sistema universitario, un Coordinador es responsable de varios Profesores; cada Profesor es responsable de uno o más Grupos. El patrón Accountability captura estas relaciones jerárquicas con tipos de responsabilidad parametrizables.

3. Patrón Observation (Observación)

Captura mediciones, observaciones o registros de fenómenos con valor, unidad y fecha.

Ejemplo

En un sistema de monitoreo atmosférico, cada lectura de un sensor (temperatura, NO₂, partículas PM2.5) se modela como una Observación con: tipo de fenómeno, valor numérico, unidad de medida, fecha-hora y ubicación geográfica.

4. Patrón Account (Cuenta)

Modela registros de transacciones acumulativas. Aplicable a bancos, inventarios, créditos académicos, etc.

5. Patrón Quantity (Cantidad)

Encapsula un valor numérico junto con su unidad de medida, permitiendo operaciones aritméticas seguras y conversiones.

Ejemplo de uso

En lugar de almacenar la temperatura como un double, se modela como Quantity(25.4, "°C"). Esto evita errores al mezclar Celsius con Fahrenheit y permite conversiones automáticas.

Diferencia entre patrones de análisis y de diseño

Aspecto	Patrones de análisis	Patrones de diseño (GoF)
Enfoque	Dominio del problema	Solución técnica
Audiencia	Analistas, expertos del negocio	Diseñadores y programadores
Nivel	Conceptual / lógico	Implementación
Autor referente	Martin Fowler	Gang of Four (GoF)
Ejemplo	Party, Observation	Singleton, Observer, Factory

4.5. Diagramas de Clases y Objetos

Diagrama de Clases

Concepto fundamental

El diagrama de clases es el diagrama estructural más importante de UML. Describe la estructura estática de un sistema mostrando las clases, sus atributos, métodos y las relaciones entre ellas. Es el punto de partida para el diseño y la implementación.

Notación de una clase

Cada clase se representa como un rectángulo dividido en tres compartimentos:

- Compartimento superior: nombre de la clase (en negrita).
- Compartimento intermedio: atributos con tipo de dato.
- Compartimento inferior: operaciones (métodos) con parámetros y tipo de retorno.

Modificadores de visibilidad

Símbolo	Visibilidad	Significado
+	Público	Accesible desde cualquier parte del sistema.
-	Privado	Accesible solo dentro de la misma clase.
#	Protegido	Accesible desde la clase y sus subclases.
~	Paquete	Accesible desde clases del mismo paquete.

Tipos de relaciones entre clases

Relación	Símbolo	Descripción	Ejemplo
Asociación	Línea simple	Conexión estructural entre clases.	Estudiante – Materia
Agregación	Rombo blanco	Todo-parte débil. Las partes existen sin el todo.	Departamento ◇— Profesor
Composición	Rombo negro	Todo-parte fuerte. Las partes mueren con el todo.	Factura ◆— Detalle
Herencia	Triángulo blanco	Una clase hija extiende a la padre.	Persona ← Estudiante
Dependencia	Línea discontinua	Una clase usa a otra temporalmente.	Servicio ⋯→ Logger
Realización	Triángulo blanco discontinuo	Una clase implementa una interfaz.	Repositorio ⋯↔ IRepositorio

Multiplicidad

Indica cuántas instancias de una clase pueden estar relacionadas con instancias de otra:

- 1 → exactamente uno
- 0..1 → cero o uno (opcional)
- * o 0..* → cero o muchos
- 1..* → uno o muchos (al menos uno)
- n..m → entre n y m

Ejemplo completo: Sistema de Inscripciones

Escenario

Un estudiante puede inscribirse en varias materias durante un semestre. Cada materia es impartida por un profesor y tiene cupo limitado. Los profesores pueden impartir varias materias.

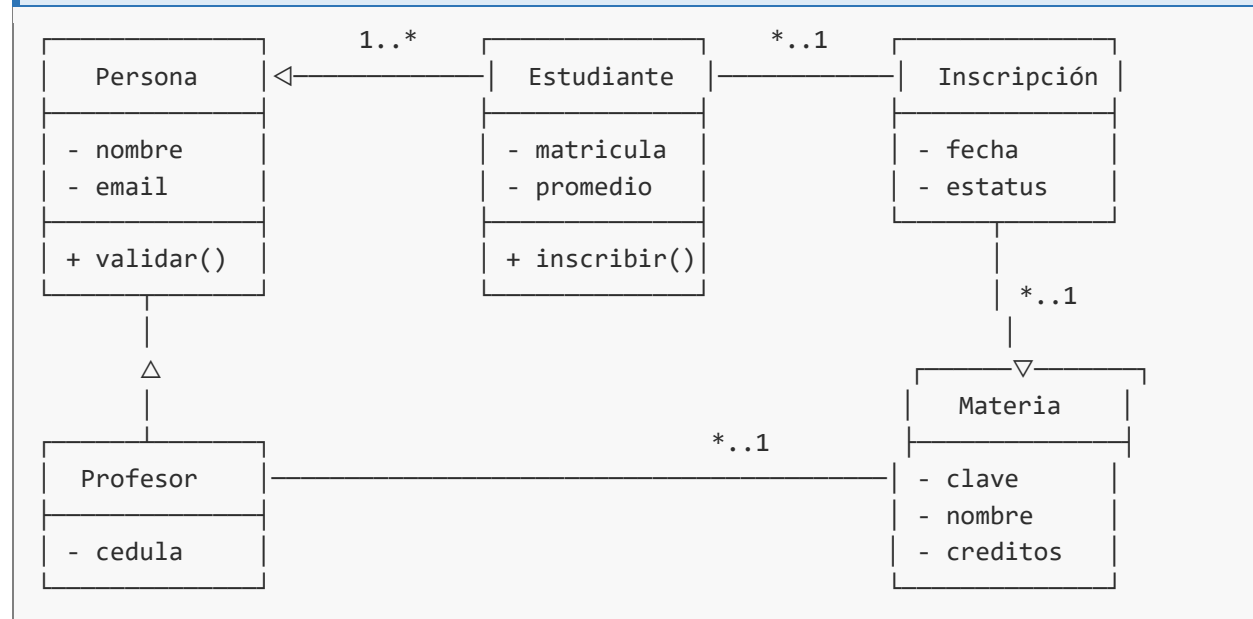


Diagrama de Objetos

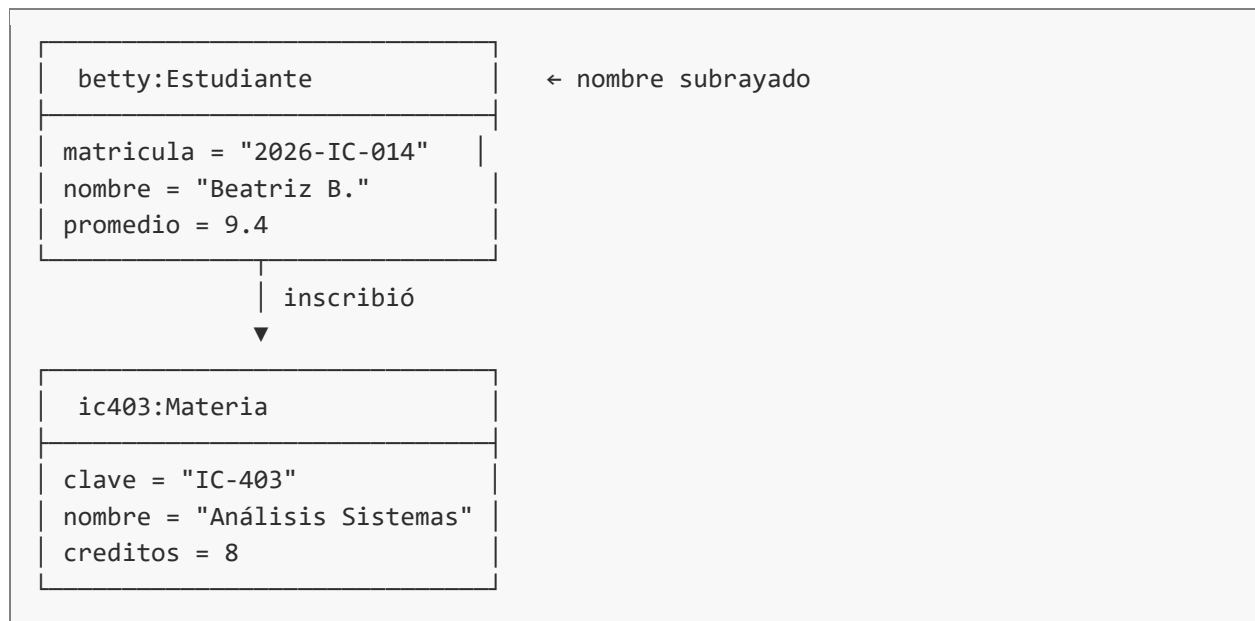
Concepto fundamental

El diagrama de objetos representa una fotografía del sistema en un instante específico de tiempo. Muestra instancias concretas de las clases con sus valores reales. Es útil para mostrar ejemplos del modelo de clases.

Diferencias clave con el diagrama de clases

Aspecto	Diagrama de clases	Diagrama de objetos
Representa	Tipos (plantillas)	Instancias (ejemplares concretos)
Nombre	Nombre de la clase	nombreObjeto:Clase (subrayado)
Atributos	Tipos de dato	Valores reales
Cantidad	Una clase por concepto	Múltiples objetos por clase

Ejemplo de diagrama de objetos



4.6. Diagramas de Actividades y Carriles

Concepto fundamental

El diagrama de actividades modela flujos de trabajo, procesos de negocio y la lógica algorítmica de un sistema. Es similar a un diagrama de flujo tradicional pero con notación UML estandarizada y mayor expresividad, ya que soporta concurrencia, decisiones complejas y sincronización.

¿Cuándo usarlo?

Cuando se necesita modelar el flujo de un caso de uso, describir un proceso de negocio, especificar un algoritmo complejo, o documentar la lógica de un método antes de implementarlo.

Elementos del diagrama de actividades

Elemento	Notación	Significado
Nodo inicial	Círculo negro relleno	Punto donde inicia el flujo.
Nodo final	Círculo negro con anillo	Punto donde termina el flujo.
Actividad	Rectángulo redondeado	Acción o tarea a realizar.
Decisión	Rombo (1 entrada, varias salidas)	Bifurcación condicional.
Unión	Rombo (varias entradas, 1 salida)	Junta caminos alternativos.
Bifurcación	Barra horizontal gruesa	Inicia ejecución paralela (fork).
Sincronización	Barra horizontal gruesa	Espera y une hilos paralelos (join).
Flujo	Flecha	Indica la secuencia de ejecución.
Objeto	Rectángulo	Datos que fluyen entre actividades.

Diagrama de Carriles (Swimlanes)

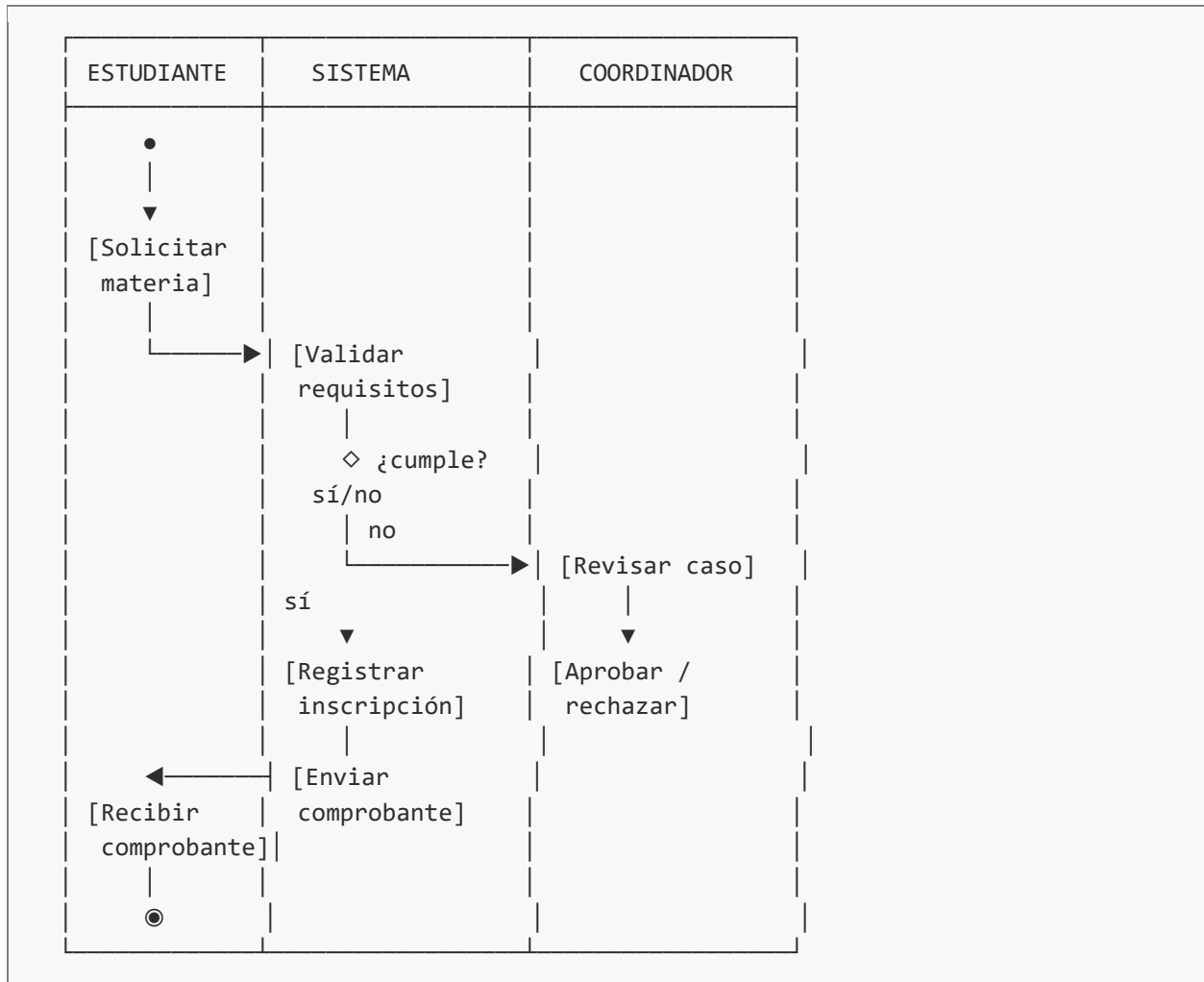
Los carriles dividen el diagrama de actividades en columnas o filas, donde cada carril representa un responsable (actor, rol, departamento o sistema). Es una herramienta excelente para identificar quién hace qué en un proceso de negocio.

Utilidad de los carriles

Permiten visualizar transferencias de responsabilidad. Cada vez que una flecha cruza un carril, ocurre una transferencia (handoff), que suele ser un punto crítico para identificar cuellos de botella o errores en el proceso.

Ejemplo: Proceso de Inscripción a Materia

Escenario: un estudiante solicita inscripción a una materia. Intervienen el estudiante, el sistema de inscripciones y el coordinador académico.



Modelado de concurrencia

UML permite modelar procesos paralelos mediante las barras de bifurcación (fork) y sincronización (join). Por ejemplo, al procesar una inscripción, el sistema puede ejecutar en paralelo: enviar correo de confirmación, actualizar inventario de cupos y registrar en bitácora. La actividad siguiente espera a que las tres terminen (join).

4.7. Diagramas de Secuencia y Comunicación

Diagrama de Secuencia

Concepto fundamental

El diagrama de secuencia muestra la interacción entre objetos a lo largo del tiempo. Es el diagrama de interacción más utilizado, ya que enfatiza la ordenación temporal de los mensajes intercambiados durante la ejecución de un escenario.

Cuándo usarlo

Para detallar cómo se realiza internamente un caso de uso: qué objetos colaboran, qué mensajes se envían, en qué orden y con qué parámetros.

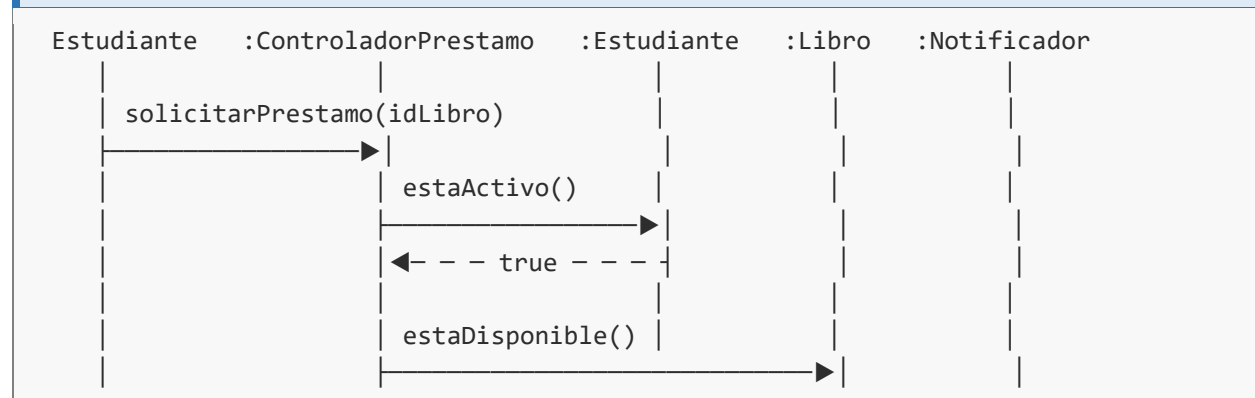
Elementos del diagrama

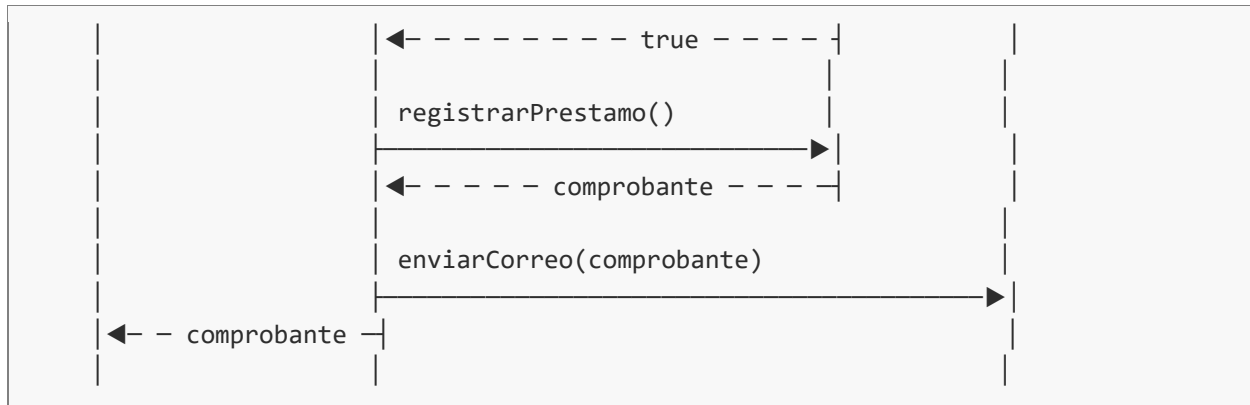
- Línea de vida (lifeline): línea vertical discontinua que representa la existencia de un objeto en el tiempo.
- Objeto/actor: encabezado de la línea de vida (rectángulo con nombre:Clase).
- Mensaje síncrono: flecha sólida con punta rellena. El emisor espera respuesta.
- Mensaje asíncrono: flecha sólida con punta abierta. El emisor no espera.
- Retorno: flecha discontinua que indica la respuesta.
- Activación: rectángulo delgado sobre la línea de vida que muestra cuándo el objeto está activo.
- Fragmentos combinados: alt (alternativas), opt (opcional), loop (bucle), par (paralelo).

Ejemplo: Caso de uso "Solicitar préstamo de libro"

Escenario

Un estudiante solicita el préstamo de un libro. El sistema valida que el estudiante esté activo, verifica disponibilidad del libro, registra el préstamo y notifica por correo.





Fragmentos combinados más comunes

Fragmento	Sintaxis	Uso
alt	alt [condición1] / [condición2]	Alternativas excluyentes (if-else).
opt	opt [condición]	Bloque opcional (if sin else).
loop	loop [condición / N veces]	Repetición del bloque.
par	par	Ejecución paralela de fragmentos.
ref	ref nombreDiagrama	Referencia a otro diagrama de secuencia.

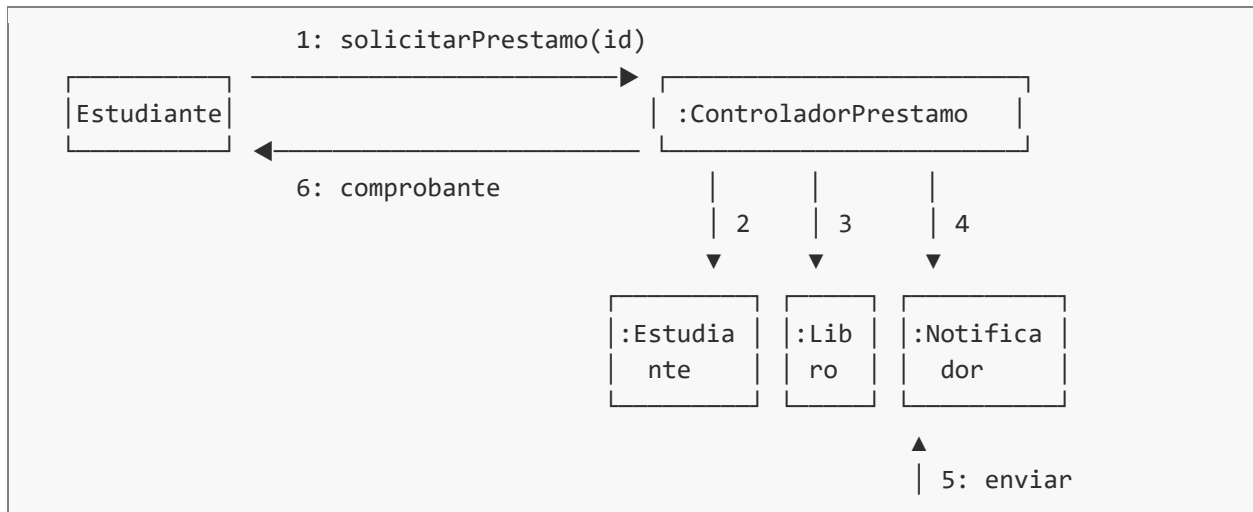
Diagrama de Comunicación

Concepto fundamental

El diagrama de comunicación (antes llamado de colaboración en UML 1.x) muestra la misma información que un diagrama de secuencia, pero enfatiza las relaciones estructurales entre los objetos en lugar de la ordenación temporal. Los mensajes se numeran para indicar su secuencia.

Cuándo preferir uno u otro

Use diagrama de secuencia cuando...	Use diagrama de comunicación cuando...
El orden temporal es lo más importante.	Las relaciones entre objetos son lo más importante.
Hay muchos mensajes en secuencia clara.	Hay pocos objetos pero muchas conexiones.
Se quiere mostrar concurrencia con par/loop.	Se quiere ver la red de colaboración.

Ejemplo: mismo caso del préstamo en notación de comunicación

La numeración 1, 2, 3... indica la secuencia. Se pueden usar números compuestos (1.1, 1.2) para indicar mensajes anidados.

4.8. Diccionario de Datos y Diagrama Entidad-Relación

Diccionario de Datos

Concepto fundamental

El diccionario de datos es un repositorio centralizado que contiene la descripción detallada de todos los elementos de información del sistema: entidades, atributos, relaciones, flujos de datos, almacenes y procesos. Es la fuente única de verdad sobre el significado de los datos.

Importancia

Sin un diccionario de datos, diferentes miembros del equipo pueden interpretar de manera distinta los mismos términos del negocio. El diccionario garantiza consistencia semántica a lo largo del proyecto.

Información que debe contener cada entrada

- Nombre del elemento (entidad, atributo, etc.).
- Descripción funcional (qué representa en el negocio).
- Tipo de dato (entero, cadena, fecha, booleano, etc.).
- Longitud o rango de valores válidos.
- Reglas de validación (formato, restricciones).
- Obligatoriedad (requerido / opcional).
- Valor por defecto.
- Origen del dato (capturado, calculado, importado).
- Relaciones con otros elementos.
- Ejemplos de valores válidos.

Ejemplo de entradas del diccionario de datos

Entidad: ESTUDIANTE

Atributo	Tipo	Longitud	Descripción	Restricciones
matricula	VARCHAR	10	Identificador único	PK, obligatorio, único
nombre	VARCHAR	100	Nombre completo	Obligatorio
fecha_nacimiento	DATE	—	Fecha de nacimiento	Mayor de 16 años
correo	VARCHAR	80	Correo institucional	Único, formato @uaemex.mx

Atributo	Tipo	Longitud	Descripción	Restricciones
promedio	DECIMAL	4,2	Promedio general	Entre 0.00 y 10.00
activo	BOOLEAN	—	Estatus del estudiante	Default: true

Diagrama Entidad-Relación (ER)

Concepto fundamental

El modelo Entidad-Relación, propuesto por Peter Chen en 1976, es una herramienta de modelado conceptual de bases de datos. Describe el universo de discurso mediante entidades, atributos y relaciones, sin entrar en detalles de implementación.

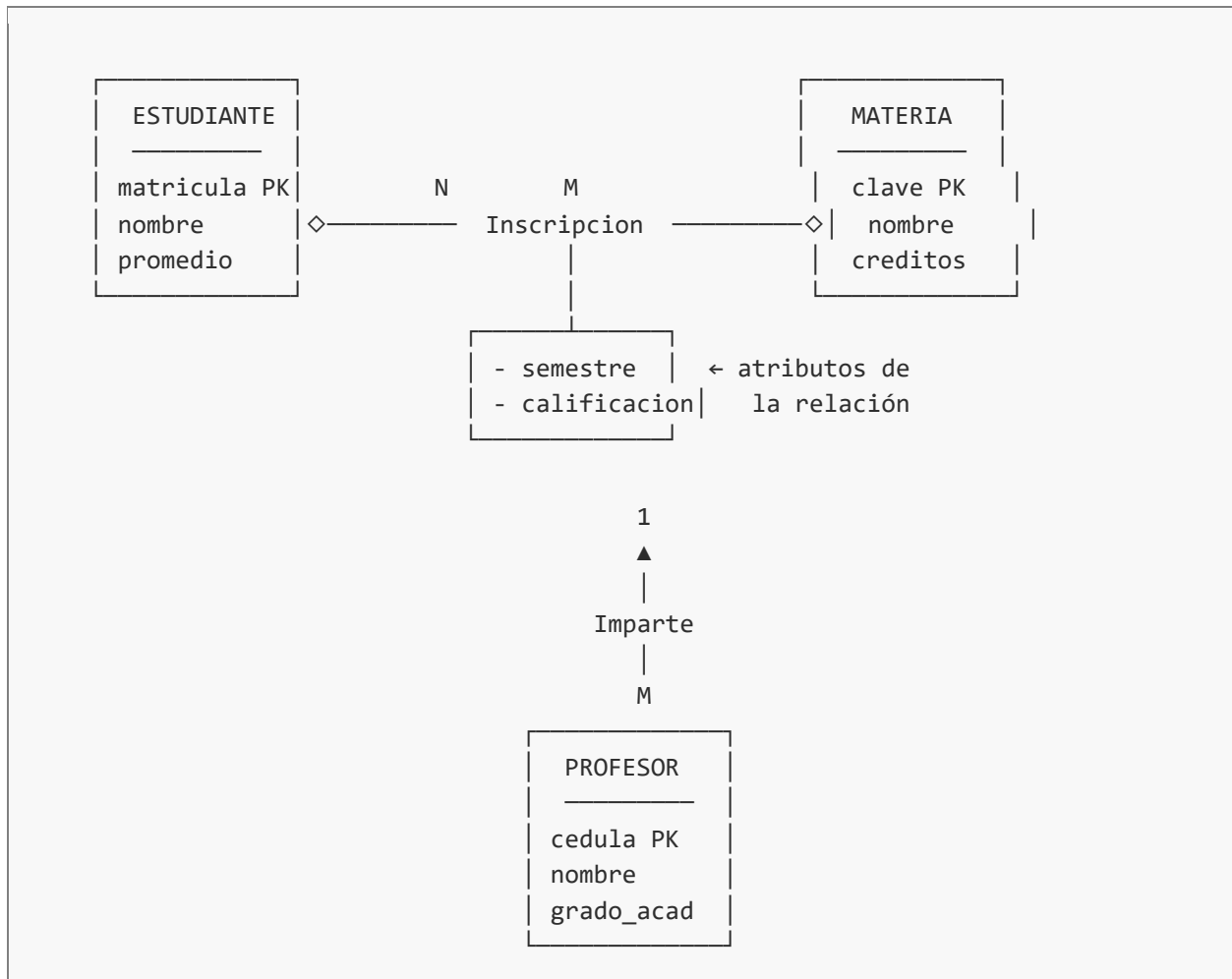
Elementos del modelo ER

Elemento	Notación clásica (Chen)	Descripción
Entidad	Rectángulo	Objeto del mundo real distinguible.
Entidad débil	Rectángulo doble	Existe en función de otra entidad.
Atributo	Óvalo	Propiedad de una entidad.
Atributo clave	Óvalo con texto subrayado	Identifica unívocamente la entidad.
Atributo multivaluado	Óvalo doble	Puede tener varios valores.
Atributo derivado	Óvalo discontinuo	Se calcula a partir de otros.
Relación	Rombo	Asociación entre entidades.
Cardinalidad	1, N, M sobre la línea	Cuántas instancias participan.

Cardinalidades

- Uno a uno (1:1): cada instancia de A se relaciona con exactamente una de B.
- Uno a muchos (1:N): una instancia de A se relaciona con varias de B.
- Muchos a muchos (N:M): varias instancias de A se relacionan con varias de B.

Ejemplo completo: Modelo ER de Sistema Escolar



Diferencia entre ER y diagrama de clases

Aspecto	Modelo Entidad-Relación	Diagrama de Clases UML
Propósito	Modelar bases de datos	Modelar estructura del software
Comportamiento	No incluye métodos	Sí incluye operaciones
Notación	Chen, Crow's foot, IDEF1X	UML estandarizado
Herencia	Limitada (especialización)	Soporte completo
Encapsulamiento	No aplica	Visibilidad (+, -, #, ~)

4.9. Estudio de Factibilidad

Concepto fundamental

El estudio de factibilidad es la evaluación formal que determina si un proyecto de software es viable de desarrollar considerando los recursos disponibles, las restricciones técnicas, los costos involucrados y los beneficios esperados. Se realiza antes de comprometer recursos significativos al proyecto.

Pregunta central

¿Vale la pena construir este sistema? El estudio de factibilidad responde considerando aspectos técnicos, económicos, operativos, legales y de tiempo.

Tipos de factibilidad

1. Factibilidad técnica

Evalúa si la organización cuenta con la tecnología, infraestructura y conocimientos necesarios para desarrollar y operar el sistema.

Aspectos a evaluar:

- Disponibilidad de hardware y software requeridos.
- Compatibilidad con sistemas existentes.
- Capacidades del equipo de desarrollo (lenguajes, frameworks).
- Escalabilidad y rendimiento esperados.
- Madurez de las tecnologías propuestas.

2. Factibilidad económica

Analiza si los beneficios del proyecto justifican la inversión requerida. Es el tipo de factibilidad más utilizado para tomar la decisión final.

Costos a considerar:

- Costos de desarrollo (sueldos, herramientas, capacitación).
- Costos de hardware e infraestructura.
- Costos de licencias de software.
- Costos de operación y mantenimiento.
- Costos de migración de datos.

Beneficios a considerar:

- Reducción de costos operativos.
- Aumento de ingresos.
- Mejora en la productividad.
- Reducción de errores y reprocesos.
- Mejora en la satisfacción del cliente.

Técnicas de análisis económico

Técnica	Fórmula	Interpretación
ROI (Retorno de Inversión)	$(\text{Beneficio} - \text{Costo}) / \text{Costo} \times 100$	Porcentaje de ganancia sobre la inversión.
VAN (Valor Actual Neto)	$\sum \text{Flujos} / (1+r)^n - \text{Inversión}$	Si VAN > 0, el proyecto es rentable.
TIR (Tasa Interna de Retorno)	Tasa donde VAN = 0	Rentabilidad del proyecto en porcentaje.
Punto de equilibrio	$\text{Costos fijos} / \text{Margen unitario}$	Momento en que beneficios igualan costos.
Período de recuperación	$\text{Inversión} / \text{Beneficio anual}$	Tiempo para recuperar la inversión.

3. Factibilidad operativa

Evalúa si el sistema será aceptado y utilizado efectivamente por los usuarios y la organización.

- ¿Los usuarios aceptarán el cambio?
- ¿Se requiere capacitación intensiva?
- ¿Existen resistencias culturales u organizacionales?
- ¿La estructura organizacional soporta el nuevo sistema?
- ¿Los procesos de negocio deben rediseñarse?

4. Factibilidad legal

Verifica que el proyecto cumpla con todas las normativas, leyes y regulaciones aplicables.

- Ley Federal de Protección de Datos Personales (México).
- Normas oficiales mexicanas aplicables al sector.
- Licencias de software (open source, propietarias).
- Derechos de autor y propiedad intelectual.
- Regulaciones del sector (educativo, financiero, salud).

5. Factibilidad de tiempo (calendario)

Determina si el proyecto puede completarse dentro de los plazos requeridos por la organización.

- Estimación realista de duración del proyecto.
- Recursos humanos disponibles y su asignación.
- Dependencias con otros proyectos.
- Hitos críticos y fechas límite.

Ejemplo: Estudio de Factibilidad para Sistema de Inscripciones UAEMéx

Contexto

La Facultad de Ingeniería desea desarrollar un sistema web para gestionar inscripciones, sustituyendo el proceso manual actual que genera filas, errores en cupos y demoras en la generación de actas.

Análisis técnico

- Infraestructura: la universidad cuenta con servidores institucionales adecuados.
- Tecnología propuesta: Python/Django + PostgreSQL, todas ampliamente probadas.
- Equipo: existen 3 desarrolladores con experiencia en el stack.
- Conclusión: técnicamente factible.

Análisis económico

Concepto	Monto (MXN)
Desarrollo (6 meses, 3 personas)	\$540,000
Hosting y licencias (1er año)	\$45,000
Capacitación de usuarios	\$25,000
Mantenimiento anual	\$80,000
Total inversión inicial	\$610,000
Ahorro anual estimado (papelería, horas-hombre)	\$420,000
Período de recuperación	≈ 17 meses

Análisis operativo

- Los estudiantes ya usan plataformas digitales para inscripciones en otras instituciones.
- El personal administrativo requiere capacitación de 20 horas.
- Se prevé resistencia inicial en personal de mayor antigüedad.

- Conclusión: operativamente factible con plan de gestión del cambio.

Análisis legal

- Cumplimiento con LFPDPPP para datos de estudiantes.
- Uso de software open source compatible con políticas universitarias.
- Sin conflictos legales identificados.

Análisis de tiempo

- Duración estimada: 6 meses de desarrollo + 1 mes de pruebas.
- Lanzamiento previsto antes del inicio del semestre 2027-A.
- Holgura de 2 meses para imprevistos.

Conclusión del estudio

El proyecto es FACTIBLE en todas las dimensiones evaluadas. Se recomienda proceder con la siguiente fase de análisis detallado de requerimientos. Período de recuperación de la inversión: 17 meses, con ROI positivo a partir del segundo año.

Conclusiones

El análisis del sistema constituye el puente entre las necesidades del cliente y la solución de software. Las técnicas y diagramas presentados en esta unidad —UML, casos de uso, diagramas de clases, actividades, secuencia, ER y estudios de factibilidad— forman un conjunto coherente que permite al ingeniero modelar la complejidad del mundo real de manera estructurada y comunicable.

Cada diagrama tiene un propósito específico: los casos de uso capturan la funcionalidad esperada; los diagramas de clases y objetos modelan la estructura; los de actividades y carriles describen procesos; los de secuencia y comunicación detallan interacciones; el modelo ER y el diccionario de datos formalizan la información persistente; y el estudio de factibilidad justifica la viabilidad del proyecto.

Dominar estas herramientas es esencial para el ingeniero en computación, ya que permiten reducir ambigüedades, anticipar problemas, facilitar el trabajo en equipo y producir software de mayor calidad. Más importante aún, estos modelos son la base sobre la que se construyen las fases posteriores de diseño, implementación y mantenimiento del sistema.

Referencias bibliográficas

- Booch, G., Rumbaugh, J. y Jacobson, I. (2006). El Lenguaje Unificado de Modelado. Pearson Addison-Wesley.
- Fowler, M. (2003). UML Distilled: A Brief Guide to the Standard Object Modeling Language (3rd ed.). Addison-Wesley.
- Fowler, M. (1997). Analysis Patterns: Reusable Object Models. Addison-Wesley.
- Larman, C. (2004). Applying UML and Patterns (3rd ed.). Prentice Hall.
- Pressman, R. S. y Maxim, B. R. (2020). Ingeniería de software: un enfoque práctico (9ª ed.). McGraw-Hill.
- Sommerville, I. (2021). Ingeniería de software (10ª ed.). Pearson.
- Chen, P. P. (1976). The Entity-Relationship Model: Toward a Unified View of Data. ACM Transactions on Database Systems.
- Object Management Group (2017). OMG Unified Modeling Language Specification, Version 2.5.1.